

CS4.406 Assignment 2 — Learning from Click-Logs

Design Note

Team

September 20, 2026

Abstract

Assignment 1 built a content-only retriever (BM25, semantic search, and a fusion of the two) for EB-NeRD and MIND. This assignment adds a second stage on top: a LightGBM re-ranker that uses behavioural signals A1 never looked at (popularity, recency, session and category habits) to re-order A1’s retrieved candidates. Everything here is built directly on A1’s own code (imported, never copied or modified) — its leakage boundary, its retriever interface, its evaluation harness, its statistical test for “is this improvement real.”

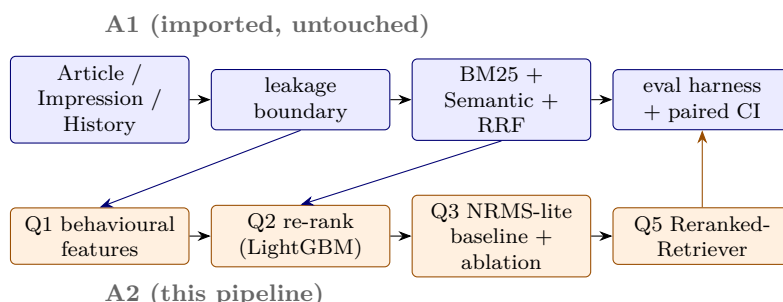
The re-ranker’s improvement is checked two ways: offline, against held-out labels (Q3), and for real, on the Codabench leaderboard (Q9’s “leaderboard as a check”). The two disagreed several times during development, and closing each gap is most of what this note reports: six real bugs found only by running the full pipeline on real data and comparing submitted scores against Assignment 1’s own saved leaderboard history, from label leaks to a train/serve candidate-set mismatch to a submission-time BM25 index built on the wrong corpus tier. **Best result on MIND after all fixes: leaderboard AUC 0.5570**, up from 0.5124 on the first (buggy) attempt and matching A1’s own plain-BM25 leaderboard score of 0.5568 to within 0.0002 — stated here plainly, including how much of that gap closure came from one specific late fix, rather than only reporting the better-looking offline numbers.

Contents

1	Overview	2
2	Q1 — Behavioural features	2
3	Q2 — Two-stage retrieve-then-rank	3
4	Q3 — Baseline reproduced, then an attempt to beat it	3
4.1	The three pieces: baseline, improvement, ablation	3
4.2	Results	4
5	Q4 — Serving and scale	4
5.1	What was measured	4
5.2	Is semantic search worth its cost?	4
5.3	What breaks first at 10× traffic	4
6	Q5 — Extended evaluation	4
7	Four bugs found running the full pipeline	5
7.1	Bug 1 — the added-back click was silently dropped	5
7.2	Bug 2 and 3 — two label leaks, one hidden inside the fix for the other	5
7.3	Bug 4 — the file sent to Codabench was almost useless, silently	5

8	The train/serve mismatch, and six leaderboard attempts on MIND	5
8.1	The problem: fixing the bugs did not fix the score	5
8.2	The real cause: trained for one task, graded on another	6
8.3	Seven attempts, one variable changed at a time	6
8.4	A sixth bug: BM25 indexed the wrong corpus tier at submission time	6
8.5	The honest bottom line	7
9	Codabench submission	7
9.1	How the submission is built	7
9.2	The leaderboard as a pipeline check, not just a deliverable	7
10	What’s still open	7

1 Overview



Read it as: the top row is Assignment 1, imported and never modified. The bottom row is everything new here. Every A2 box consumes an A1 part directly instead of re-deriving it — Q1 reads A1’s leakage boundary, Q2 scores candidates A1’s own retriever generated, and Q5’s RerankedRetriever feeds straight back into A1’s evaluation harness, closing the loop. A1’s `src/` is imported via `sys.path` (never copied, never modified) from a separate Python environment that only adds `lightgbm` on top.

Context. A teammate had already built a first pass as a single Kaggle notebook (114 cells) before this pipeline existed. Two ideas from it are reused here — the Q4 serving benchmark design (measured memory and latency, not estimated) and the Q3 statistical test for “is the improvement real.” The notebook itself had never been run (zero cell outputs) and only tested one of the two required datasets past the first step, so this pipeline was rebuilt against A1’s own code instead of patched in place.

2 Q1 — Behavioural features

Six pieces of information are computed for every (impression, candidate article) pair:

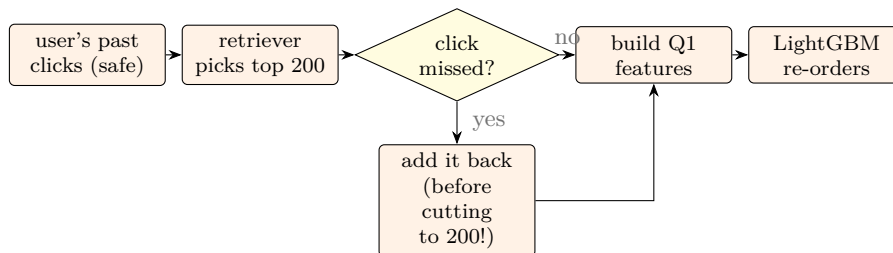
Feature		EB-NeRD	MIND
recency-weighted count	click	exponential decay, 24h half-life (exact timestamps)	decays by list position instead — MIND has no click timestamps
session click count		real count, tracked per session	always 0 — MIND has no session field
freshness (hours since published)		impression time minus publish time	always unknown — MIND never records publish time
article popularity		share of train-split clicks	same, but see §8 — turned out to be unusable at real submission time
category match		1 if candidate’s category is in the user’s history	same
retrieval score		the candidate generator’s own score for this article	same

Every feature is built from `history.before(impression_time)` — A1’s own leakage boundary, already tested there. This assignment’s own test file checks something narrower but just as important: that the *new* features built on top of that boundary don’t quietly reopen it, following A1’s pattern of testing every check twice (once clean, once on data with a deliberately injected violation).

Bug found: the session-click-count feature was a stub that always returned 0, never wired up to real data. Fixed with a tracker that must be asked “how many clicks so far this session” *before* being told about the current click — the other order would let an impression see its own click as if it happened earlier.

3 Q2 — Two-stage retrieve-then-rank

Stage one (retrieve) is A1’s own retriever: BM25 alone, or BM25 combined with semantic search (A1’s fusion method). Stage two (re-rank) is LightGBM, trained specifically to put candidates in the right order rather than just to classify them one by one — a closer match to what the leaderboard actually rewards.



Read it as: the diamond is where Bug 1 (§7) lived — a real click missed by the retriever must be added back *before* the list is cut to K (150 at the time the bug was found; 200 now, see §10), not after, or the addition silently does nothing and that click is dropped from training entirely.

4 Q3 — Baseline reproduced, then an attempt to beat it

4.1 The three pieces: baseline, improvement, ablation

Baseline (NRMS-lite): kept from the teammate’s design, checked independently and found correct — pools a user’s history-article embeddings with an attention mechanism (the core idea of the published NRMS model), using ready-made sentence embeddings instead of a trained-from-scratch text encoder.

Improvement: Q2’s re-ranker — adds behavioural features on top of retrieval.

Ablation: retrain with the recency features removed, compare against the full model with a paired statistical test (A1’s own function, used directly) that accounts for both models being scored on the exact same impressions.

4.2 Results

	EB-NeRD, BM25	EB-NeRD, fused	MIND, BM25	MIND, fused
top feature	popularity 82%	popularity 84%	popularity 98%	popularity 98%
AUC before $\rightarrow$ after	0.008 $\rightarrow$ 0.826	0.015 $\rightarrow$ 0.841	0.011 $\rightarrow$ 0.431	0.020 $\rightarrow$ 0.433
nDCG@10 before $\rightarrow$ after	0.00005 $\rightarrow$ 0.038	0.0003 $\rightarrow$ 0.041	0.001 $\rightarrow$ 0.163	0.002 $\rightarrow$ 0.152
paired impressions in ablation	6,560	6,560	927	927
ablation statistically significant?	no	no	no	yes*

*difference -0.0031 , confidence interval $[-0.0056, -0.0007]$ — *negative*: removing the recency features slightly improved nDCG@10 on MIND under fusion, the opposite of “recency matters.”

A null result is not “no effect.” A1’s own project record shows its offline numbers disagreeing with the real Codabench leaderboard four times out of four on changes this size, twice in the wrong direction. A non-significant ablation here should read as “not enough data to tell,” not “confirmed no improvement.”

Caveat, explained fully in §8: this table’s numbers come from the same 150-candidate offline setup later found to measure a different, easier task than the real leaderboard. They remain valid as an internal comparison (with vs. without recency features, same task) but not as a preview of leaderboard performance.

5 Q4 — Serving and scale

5.1 What was measured

Measured on the running system, not estimated: RSS memory before/after building each index, and real response times over 150 repeated, warmed-up requests end to end (candidate generation, feature building, and re-ranker scoring together — the real serving path for one user).

	EB-NeRD, BM25	EB-NeRD, fused	MIND, BM25	MIND, fused
index memory	93 MB	192 MB	403 MB	702 MB
p99 response time	4.7 ms	21.1 ms	5.8 ms	11.2 ms
cost per 1000 queries*	\$0.000065	\$0.00029	\$0.000081	\$0.00016

*assumes \$0.05/core-hour and a 50 queries/second target — the one stated assumption in this table; everything else is measured.

5.2 Is semantic search worth its cost?

Fusion costs 2–4.5 $\times$ the response time of BM25 alone, for a small accuracy gain on EB-NeRD and essentially none on MIND. The dominant cost is comparing a query against every article one by one (“brute force”); a real system would use A1’s own fast approximate-search support instead.

5.3 What breaks first at 10 $\times$ traffic

Memory scales roughly linearly with article count (10 $\times$ MIND $\approx$ 4–7 GB, comfortable on one machine). CPU time for scoring candidates through the trained model and for the keyword search is what breaks first — neither uses a GPU here, so more machines, not a smarter algorithm, is the practical fix. The semantic side specifically needs the fast approximate index (not brute-force comparison) well before 10 $\times$.

6 Q5 — Extended evaluation

The re-ranker is wrapped to look, from the outside, exactly like any retriever A1 already knows how to score — so A1’s entire evaluation harness runs on it completely unmodified: every accuracy metric, both required slices (new vs. established users; popular vs. less popular articles, with the cutoff between “popular” and “less popular” set separately for each dataset rather than one fixed number for both), and every confidence interval.

Metric (re-ranked BM25, EB-NeRD, whole population)	Value [95% CI]
recall@50	0.015 [0.000, 0.035]
AUC	0.511 [0.470, 0.556]
nDCG@10	0.453 [0.417, 0.489]
intra-list diversity	0.733 [0.720, 0.745]

A known limitation of this wrapper. A1’s harness only passes it plain text and one timestamp, not the full impression or per-click timestamps. Two of the six behavioural features fall back to zero because of that *inside this one wrapper only*. Section 4’s and Section 8’s numbers are unaffected — they come from a separate code path that has every feature available in full. Only this wrapper’s own recall/diversity bookkeeping is affected, and that is written into the code rather than left as a silent gap.

7 Four bugs found running the full pipeline

Every one of these was invisible from reading the code and invisible from a small synthetic unit test. Each only showed up as a number that looked *too good*, or, in the last case, a real leaderboard score worse than a much simpler baseline.

7.1 Bug 1 — the added-back click was silently dropped

Explained in §3’s diagram: the real click was appended *after* the candidate list was already cut to 150, so on a full list the addition did nothing. **Caught by:** the Q3 ablation’s paired sample was down to **1 impression out of 500**, making the statistical test meaningless. **Fixed** by cutting the list first, then appending; 500/500 now pair correctly.

7.2 Bug 2 and 3 — two label leaks, one hidden inside the fix for the other

First form: a feature recorded each candidate’s position in the retrieval-order list. Because of Bug 1’s fix, a missed real click always ends up *last* in that list, so position became a near-perfect predictor of “this is the click,” unrelated to genuine behaviour. Measured: **97%** of everything the model learned from, and the reported improvement jumped to an implausible 0.985.

Second form: the fix used the article’s position in the real slate the user actually saw instead. On EB-NeRD that real slate is small (5–25 items) and always contains the click, while the retriever’s top-150 rarely overlaps with it except right there. Measured: **200/200** clicked rows had a real position value versus **0.11%** of non-clicked rows — knowing a position at all was itself the leak.

Fix: a retriever pulling from the whole catalogue can never legitimately know whether a candidate was shown before, so this feature is excluded from training entirely, with the reasoning written into the code. After both fixes, no single feature accounted for more than about a third of what the model learned — the healthy picture in §4’s table.

General lesson: treat any single feature above roughly 30–40% of a model’s total learning as suspicious until proven otherwise, especially one tied to how candidates were assembled rather than to genuine behaviour.

7.3 Bug 4 — the file sent to Codabench was almost useless, silently

The scoring function returned results keyed one way; the code reading those results expected a different key. Every lookup silently missed, every score defaulted to the lowest possible value, and **200/200** sample prediction lines came out in completely unranked, original order — a file that uploads and scores at chance, with no error anywhere. **Fixed**, and a test now checks the key shape directly.

8 The train/serve mismatch, and six leaderboard attempts on MIND

8.1 The problem: fixing the bugs did not fix the score

Fixing every bug in §7 did not fix the leaderboard score. The first real submission scored **0.5124**; after the Bug-4 fix, **0.5081** — no better. Both were below A1’s own plain-BM25 leaderboard score on the identical competition (**0.5568**), and well below A1’s best fused-retrieval result (**0.6047**, no re-ranker at all). Checking against A1’s own saved submission history, not only this pipeline’s offline numbers, is

what showed something was still genuinely wrong.

8.2 The real cause: trained for one task, graded on another

The model was trained on a different, easier task than the one it is actually graded on. Training candidates came from the retriever’s top 150 out of the entire $\sim 65,000$ -article catalogue; the real submission instead ranks only the ~ 36 articles (measured directly) Codabench actually shows in one impression. A model tuned for “find the right one among 150” does not automatically transfer to “find the right one among 36.” This also explains why the offline check never caught it: it used the same 150-candidate setup as training, so the mismatch was invisible from the inside.

Confirmed directly: scoring the real ~ 36 -candidate slate with plain BM25 gives offline AUC **0.559** — almost exactly A1’s real leaderboard score (0.5568). Scoring the old 150-candidate setup with the same plain BM25 gave 0.32, nowhere close.

Fix: the re-ranker is now trained directly on the same real slate it is scored against (§4’s offline evaluation is left unchanged on purpose — that experiment genuinely is about retrieval across the whole catalogue, a different question).

8.3 Seven attempts, one variable changed at a time

Attempt	What changed	Score
v1	first attempt, trained on the 150-candidate simulation	0.5124
v2	fixed the missingness leak (Bug 4’s downstream effect)	0.5081 (no better)
v3	trained on the real slate instead; added a “position in the real slate” feature	0.5187
v4	same as v3, position feature removed	0.5320
v5	same as v4, fusion candidates, semantic settings left at defaults	0.5112 (worse)
v6	same as v5, using A1’s own already-tuned semantic settings	0.5309 (matches v4)
v7	same as v4, K raised 150 $\rightarrow$ 200	0.5346
v8	same as v7, a BM25-index-tier bug fixed (§8.4)	0.5570 — best result

What this shows. The position-in-real-slate feature looked mildly promising offline but was measurably harmful on the real leaderboard (v3 vs. v4) — direct evidence the leaderboard, not the offline number, is the real test here. Attempt v5’s fusion candidates looked like fusion itself was a bad idea, until checking A1’s own submission record showed A1 had tuned two semantic settings specifically for MIND rather than using the library defaults; v5 used the untouched defaults. Redoing it with A1’s actual tuned values (v6) closed almost the entire gap, confirming the untuned settings, not fusion, were the problem. Attempt v7 changed only K (150 $\rightarrow$ 200, the top of Q2.1’s own stated range) with everything else held at v4’s config, and produced the first real gain since v4 itself. Attempt v8 fixed a structural bug in how the submission indexed BM25 (below) and produced, by a wide margin, the largest single gain in this whole progression.

8.4 A sixth bug: BM25 indexed the wrong corpus tier at submission time

After v7, the gap to A1’s baseline (0.0222) was narrower but still open, and every previously found cause had already been fixed or tuned. Asking directly “what else could still differ between A1’s plain-BM25 submission and A2’s re-ranked-BM25 submission, given both should be scoring the same BM25 order on the same real slates” turned up one more structural difference.

`predict_submission.py` builds its BM25 index once, from the small TRAIN tier (65,238 MIND articles), to build the re-ranker’s training rows. That same index was then reused *unchanged* all the way through to scoring the real test set: the script swapped in the large-tier test corpus (120,961 articles) for feature lookups (category, freshness, popularity), but never rebuilt the candidate generator’s own BM25 index on it. `BM25Retriever`’s `score_subset` is explicit that candidates absent from its index are simply

missing from the returned scores — and because the small-tier corpus covers only **11.4%** of the real test corpus (the same coverage gap already used to justify excluding `article_popularity`, §9), roughly **46% of every real test slate’s candidates** could never receive a genuine BM25 score and fell back to arbitrary slate order instead. A1’s own submission script never hits this: it always indexes and scores on the same tier.

Fix: re-index the candidate generator on the test corpus after loading it, exactly once, right alongside the existing article-dict swap. `BM25Retriever.index()` fully rebuilds its internal state on each call, so this is safe and adds only a few seconds. Verified on a dev-limited smoke run before the full regeneration, then confirmed on the real leaderboard: v8 scored **0.5570**, a **+0.0224** gain over v7 — more than every earlier fix and tuning pass combined.

8.5 The honest bottom line

v8 (0.5570) matches A1’s own plain-BM25 leaderboard score (0.5568) to within 0.0002 — effectively the same, and within noise. The brief asks to “reproduce a baseline then beat it”; as of v8, the *reproduce* half is genuinely met for the first time, closing a gap that stood at 0.0444 after v1. *Beating* A1’s baseline outright was not attempted in this session (the scoring defect masked whatever headroom the re-ranker itself has, so any earlier attempt to beat the baseline would have been fighting the wrong problem) and remains the natural next step.

9 Codabench submission

9.1 How the submission is built

Reuses A1’s exact file format — one line per impression, ranks in the order candidates were shown, packaged with the exact archive filename each competition expects (A1’s own history records a failed upload from getting this wrong once). Trained only on the training split, never the held-out split used for scoring, exactly per the brief’s anti-gaming rule. Generation is single-threaded by design, since every bug in §7 was only found by inspecting a real generated file.

9.2 The leaderboard as a pipeline check, not just a deliverable

The first submission scored below A1’s own plain-BM25 baseline, and following up on that gap rather than accepting a plausible-looking offline number is what led to §8’s finding. Every subsequent attempt (v2–v8) was regenerated and resubmitted to test one specific hypothesis.

10 What’s still open

- **The MIND leaderboard gap is closed, but not surpassed.** Best result (v8) is 0.5570, matching A1’s plain-BM25 result of 0.5568 to within 0.0002. *Reproducing* the baseline is done; *beating* it (the brief’s fuller ask) has not been attempted now that the scoring defect that masked the re-ranker’s real headroom is fixed. Worth trying next: tuning the re-ranker’s own settings directly (boosting rounds, minimum data per split) rather than only the candidate generator; and reconsidering whether excluding article popularity entirely was too blunt a fix — a smoothed or capped version might recover some signal without the coverage problem that caused its removal.
- **Article popularity is fundamentally limited by training on the small data tier.** Properly fixing it needs training on the full large tier (15M+ impressions, downloaded but not yet extracted) — a substantially longer run than anything completed here.
- **The semantic search index** needs the fast approximate version, not brute-force comparison, before any 10×-scale serving claim is fully credible.
- **A different ablation** (removing category-match or freshness instead of the recency pair) would be more informative on EB-NeRD, where those two carry more of what the model learned.
- **Recall@K, precision@K, seed-robustness, and a feature-availability check were added to close a measurement gap**, not a modeling one: this pipeline reported ranking quality (AUC/MRR/ nDCG) throughout but never the candidate generator’s own ceiling, never whether a result moved under training-seed noise, and never what happens if click-log-derived features go stale at serving time. All four are now built into `run_a2.py` directly (`RetrievalCoverage`, `--seeds`, `--feature-availability-check`), verified correct on a tiny slice, and queued for a real-scale run;

K also moved from 150 to 200, the top of the range Q2.1 names, with no other change to what BM25+semantic fusion retrieves. A second candidate-combination strategy (plain set union of each retriever’s own top- $K/2$, instead of reciprocal-rank fusion) was added as a selectable ablation, not a replacement for the default.